ORCAS: Efficient Adaptor Signatures for CSI-FiSh

Ryann Cartor, **Nathan Daly**, Giulia Gaggero,
Jason LeGrow, Andrea Sanguineti, and Silvia Sconza

The Isogeny Club
3 March 2026

VIRGINIA TECH

ORCAS

One
Round
Cheating
Adaptor
Signatures

I. Introduction

Motivation & Definitions

Exchanging Cryptocurrencies

I have 100 HokieCoin. You have 100 CavCash. We both want 50 of each. We can exchange by publishing the following transactions to the blockchain:

Pay 50 CavCash
To Me. Signed:
U. Yourself

Pay 50 HokieCoin
To You. Signed:
I. Myself

Exchanging Cryptocurrencies

I have 100 HokieCoin. You have 100 CavCash. We both want 50 of each. We can exchange by publishing the following transactions to the blockchain:

Pay 50 CavCash
To Me. Signed:
U. Yourself

Pay 50 HokieCoin
To You. Signed:
I. Myself

But who will publish first?

Possible Solutions

Escrow Service

A trusted third party collects and distributes both our assets, or returns one party's assets if the other fails to post a transaction in time.

Problem: Escrow services may be expensive, unavailable, or unwanted.

Smart Contract

On-chain code acts as the trusted third party, enforcing that neither transaction can take effect until the other has also been posted.

Problem: Not all blockchains support smart contracts.

Possible Solutions

Escrow Service

A trusted third party collects and distributes both our assets, or returns one party's assets if the other fails to post a transaction in time.

Problem: Escrow services may be expensive, unavailable, or unwanted.

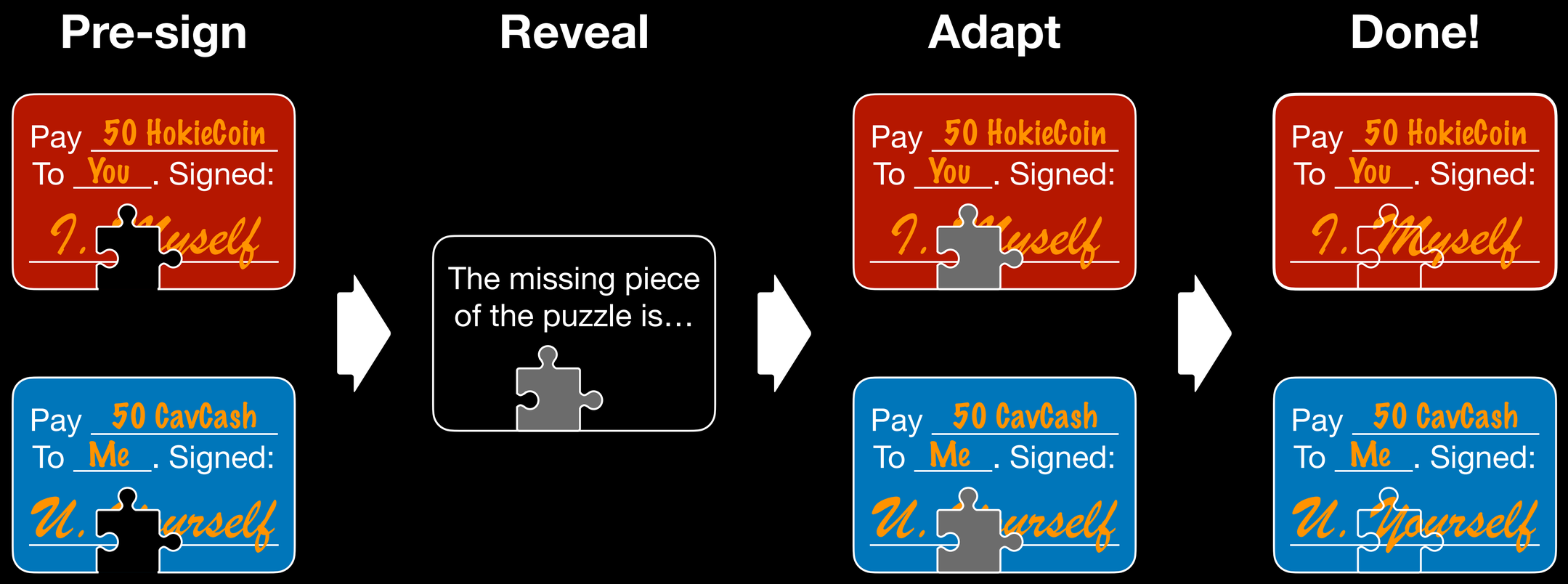
Smart Contract

On-chain code acts as the trusted third party, enforcing that neither transaction can take effect until the other has also been posted.

Problem: Not all blockchains support smart contracts.

Is there a solution using only basic, "off-the-shelf" blockchain features?

Idea: Adaptor Signatures



Signature Scheme

Algorithms

- KeyGen(1^λ) → (*sk*, *pk*)
- Sign(*sk*, *msg*) → *sig*
- Verify(*pk*, *msg*, *sig*) → 0/1

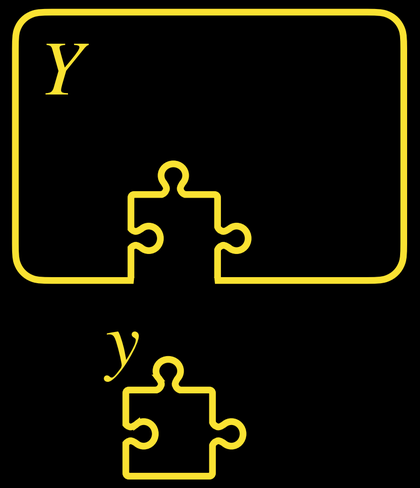
Keys

- *sk* - Secret Key (used to sign)
- *pk* - Public Key (used to verify)

Hard to…

- Create a *sig* for *pk* without knowing *sk*

Adaptor Signature Scheme



Algorithms

- KeyGen(1^λ) → (*sk*, *pk*)
- Sign(*sk*, *msg*) → *sig*
- Verify(*pk*, *msg*, *sig*) → 0/1
- RGen(1^λ) → (*Y*, *y*)
- PreSign(*sk*, *Y*, *msg*) → *psig*
- PreVerify(*pk*, *Y*, *msg*, *psig*) → 0/1
- Adapt(*y*, *psig*) → *sig*
- Extract(*sig*, *psig*, *Y*) → *y*

Keys

- *sk* - Secret Key (used to sign)
- *pk* - Public Key (used to verify)
- *Y* - Statement (use to pre-sign)
- *y* - Witness (use to adapt)

Hard to…

- Create a *sig* for *pk* without knowing *sk*
- Adapt *psig* → *sig* without knowing *y*
- Adapt *psig* → *sig* without revealing *y*

II. Schnorr

Adaptor Signatures from Discrete Log

Schnorr Sigma Protocol

Setup

- Let $\mathbb{G} = \langle g \rangle$ be a multiplicative group of prime order N.
- There is a group action of $\mathbb{Z}/N\mathbb{Z}$ on $\mathbb{G}$ given by $[a]X = g^a X$ for $a \in \mathbb{Z}/N\mathbb{Z}$ and $X \in \mathbb{G}$.
- Pick a secret key $x \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.
- Define $X_0 = e_{\mathbb{G}}$ and $X_i = [ix]X_0 = g^{ix}$.
- ***Note: we can compute X_j from X_1 without knowing x itself, by computing $X_i = X_1^i$.***

$$X_0 \xrightarrow{[x]} X_1 \xrightarrow{[x]} X_2 \quad \cdots \quad X_{N-1}$$

Schnorr Sigma Protocol

Setup

- Let $\mathbb{G} = \langle g \rangle$ be a multiplicative group of prime order N.
- There is a group action of $\mathbb{Z}/N\mathbb{Z}$ on $\mathbb{G}$ given by $[a]X = g^a X$ for $a \in \mathbb{Z}/N\mathbb{Z}$ and $X \in \mathbb{G}$.
- Pick a secret key $x \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.
- Define $X_0 = e_{\mathbb{G}}$ and $X_i = [ix]X_0 = g^{ix}$.
- ***Note: we can compute X_j from X_1 without knowing x itself, by computing $X_i = X_1^i$.***

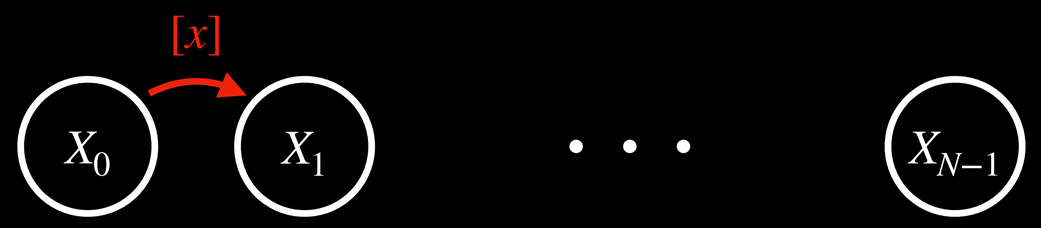
$$X_0 \xrightarrow{[x]} X_1 \xrightarrow{[x]} X_2 \quad \cdots \quad X_{N-1}$$

ASSUMPTION

The **discrete log problem** is to find x given X_0 and X_1. We assume this problem is computationally infeasible for the chosen parameters $\mathbb{G}$ and g.

Schnorr Sigma Protocol

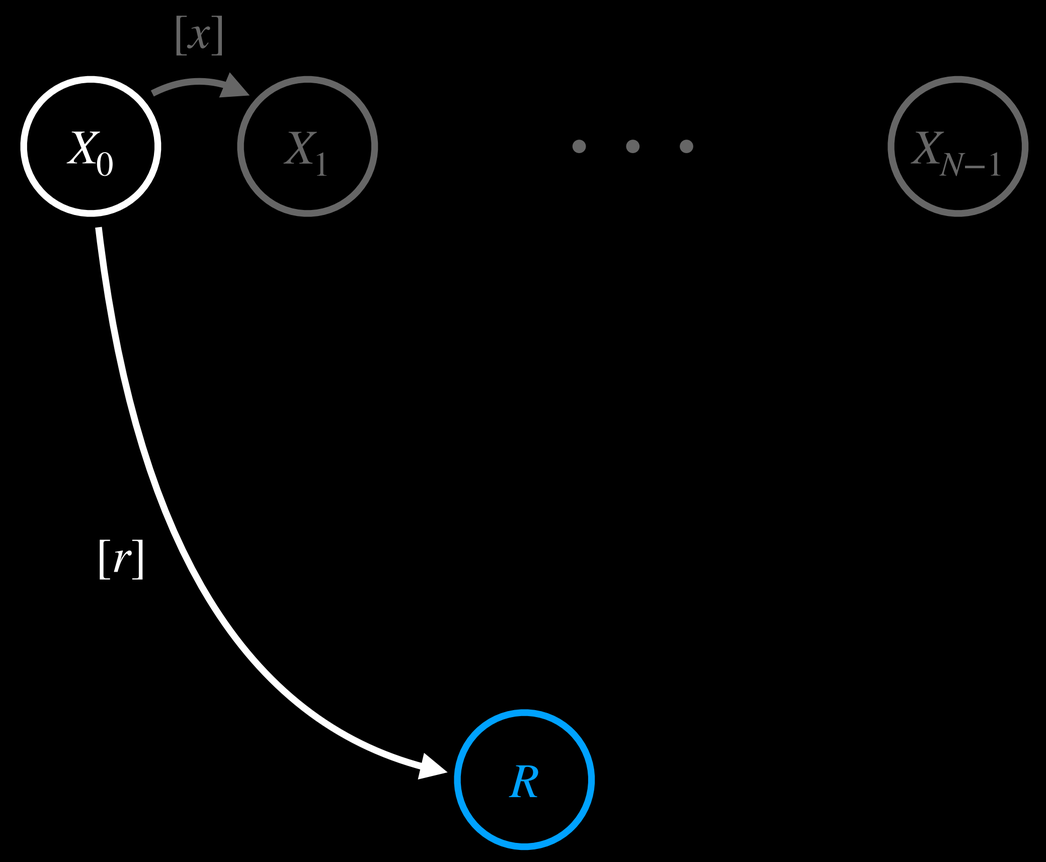
The Schnorr sigma protocol is a 3-message protocol to prove knowledge of a secret key x such that $X_1 = [x]X_0$, without revealing x.



Schnorr Sigma Protocol

The Schnorr sigma protocol is a 3-message protocol to prove knowledge of a secret key x such that $X_1 = [x]X_0$, without revealing x.

1. **Commitment.** Prover samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and publishes $R = [r]X_0 = g^r$.

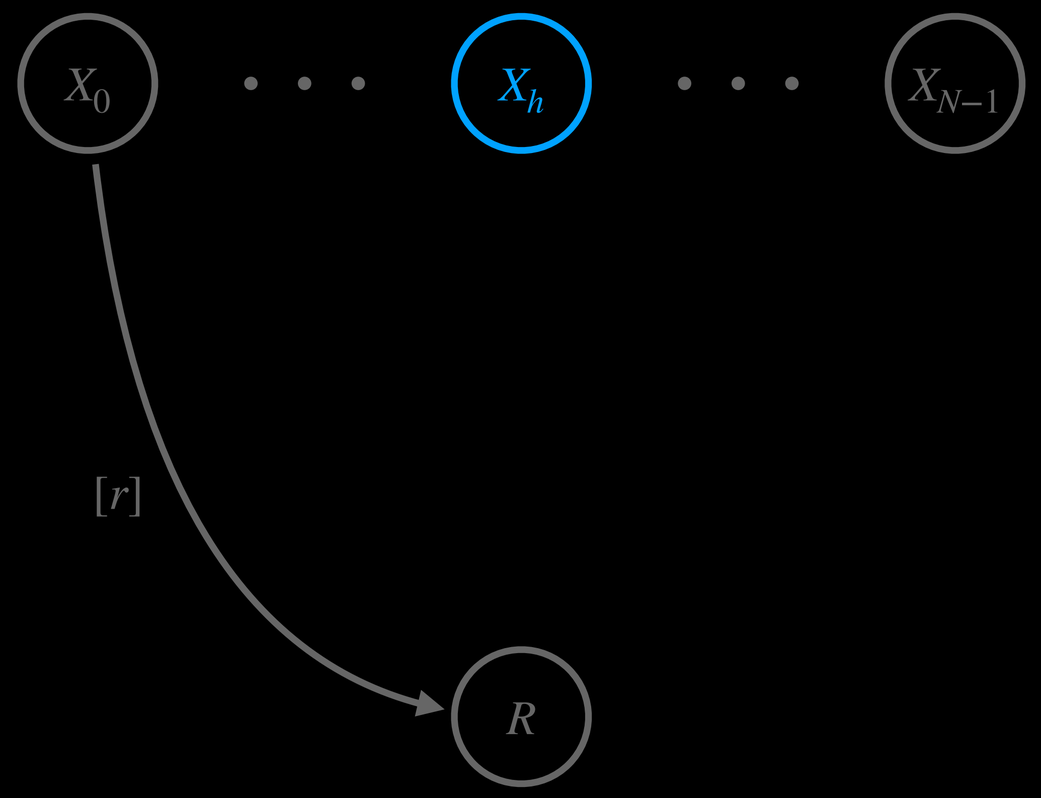


Schnorr Sigma Protocol

The Schnorr sigma protocol is a 3-message protocol to prove knowledge of a secret key x such that $X_1 = [x]X_0$, without revealing x.

1. **Commitment.** Prover samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and publishes $R = [r]X_0 = g^r$.

2. **Challenge.** Verifier publishes $h \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.



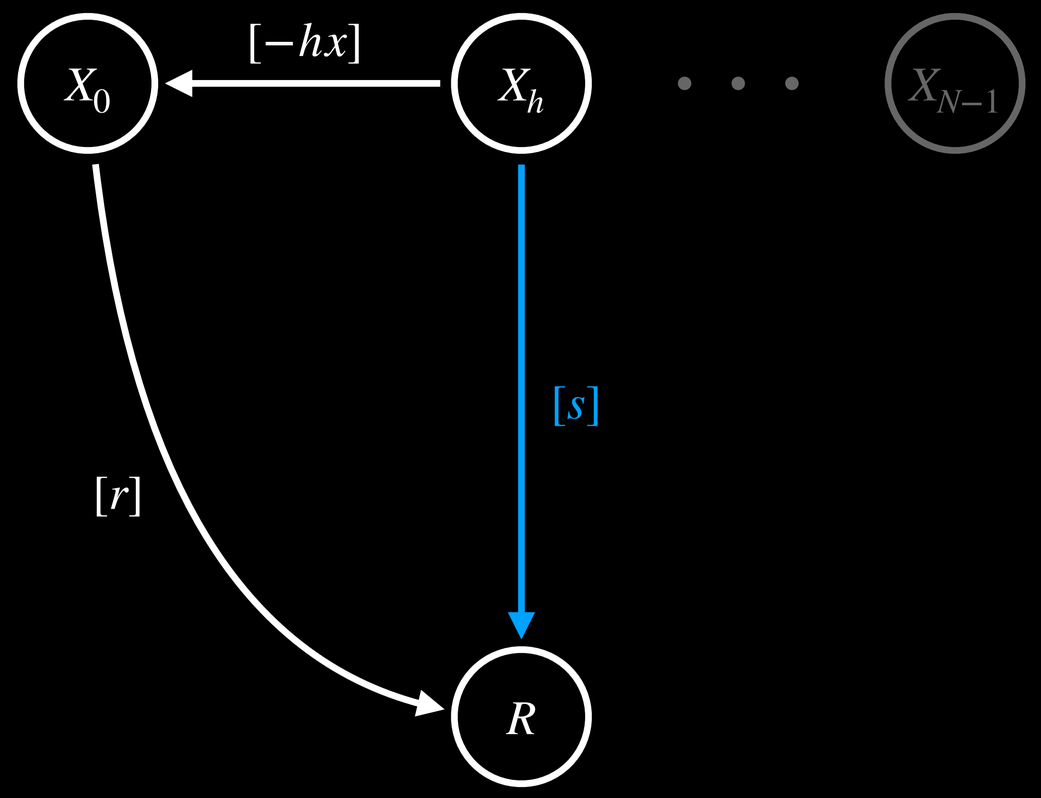
Schnorr Sigma Protocol

The Schnorr sigma protocol is a 3-message protocol to prove knowledge of a secret key x such that $X_1 = [x]X_0$, without revealing x.

1. **Commitment.** Prover samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and publishes $R = [r]X_0 = g^r$.

2. **Challenge.** Verifier publishes $h \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.

3. **Response.** Prover publishes $s = r - hx$.



Schnorr Sigma Protocol

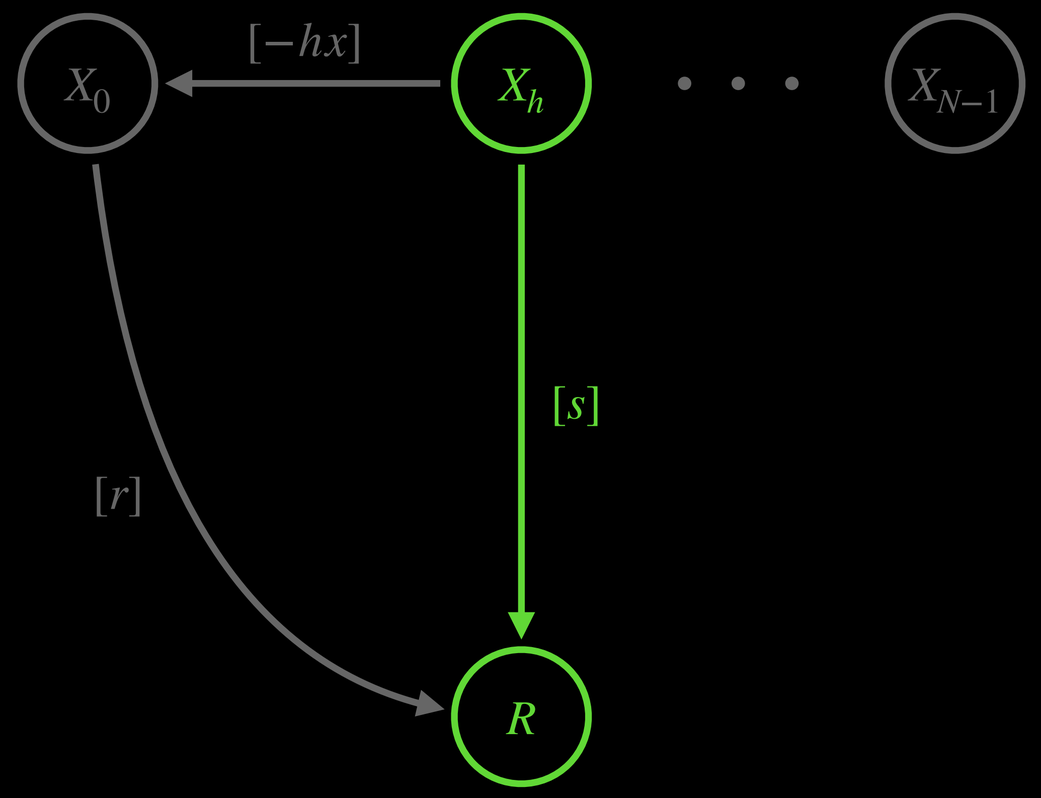
The Schnorr sigma protocol is a 3-message protocol to prove knowledge of a secret key x such that $X_1 = [x]X_0$, without revealing x.

1. **Commitment.** Prover samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and publishes $R = [r]X_0 = g^r$.

2. **Challenge.** Verifier publishes $h \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.

3. **Response.** Prover publishes $s = r - hx$.

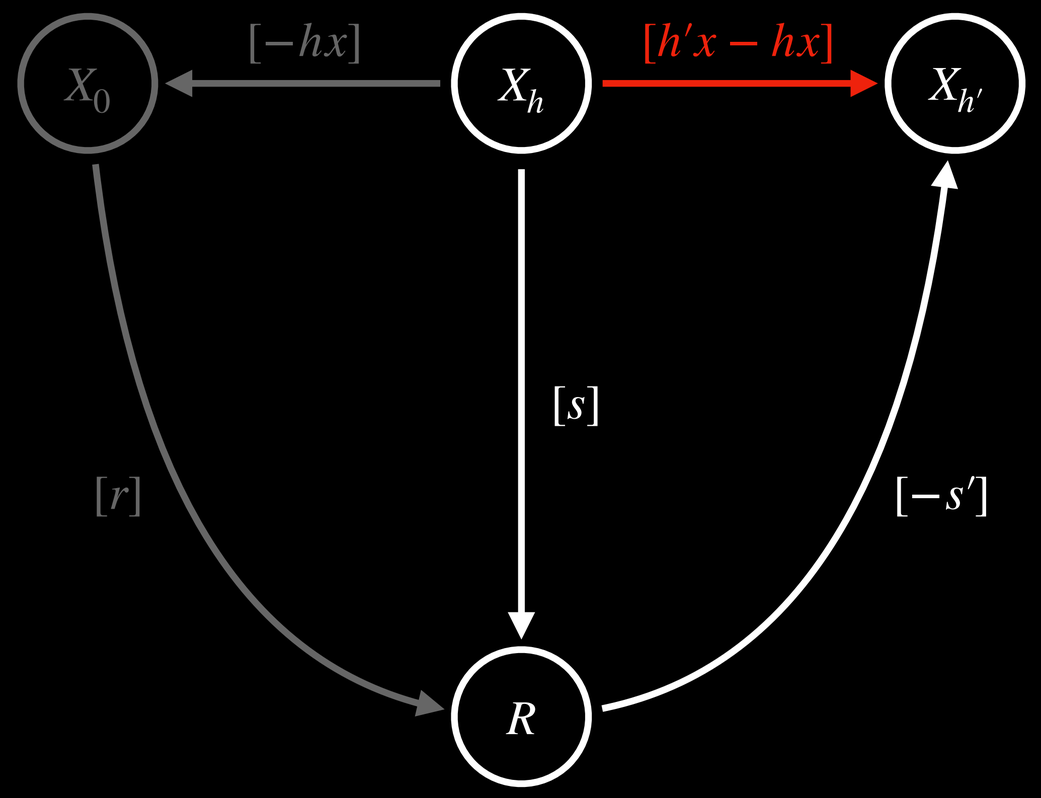
Verifier accepts if $R = [s]X_h$.



Security of Schnorr

2-Special Soundness. Suppose, for a fixed commitment R, the prover is able to give acceptable responses s, s' for two different challenges $h \neq h'$. Then we can compute

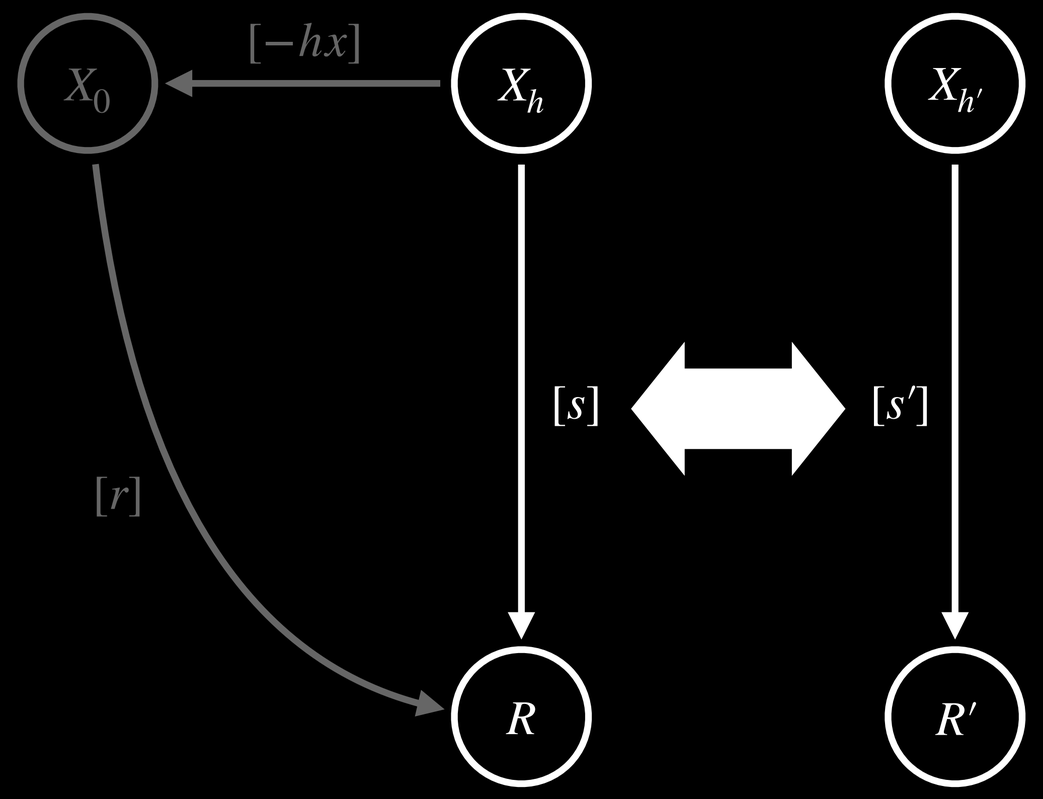
$$\frac{s - s'}{h' - h} = \frac{r - hx - r + h'x}{h' - h} = x.$$



Conclusion: *No matter its choice of R, a prover that does not know x can produce a correct response with probability at most $1/N$.*

Security of Schnorr

Honest-Verifier Zero Knowledge. Without knowing x, we can sample $h', s' \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and compute $R' = [s']X_{h'}$ to obtain a transcript (R', h', s') distributed identically to an honest transcript (R, h, s).



Conclusion: *The verifier learns no new information about x, since it could have simulated the interaction on its own.*

Recall...

Schnorr Sigma Protocol

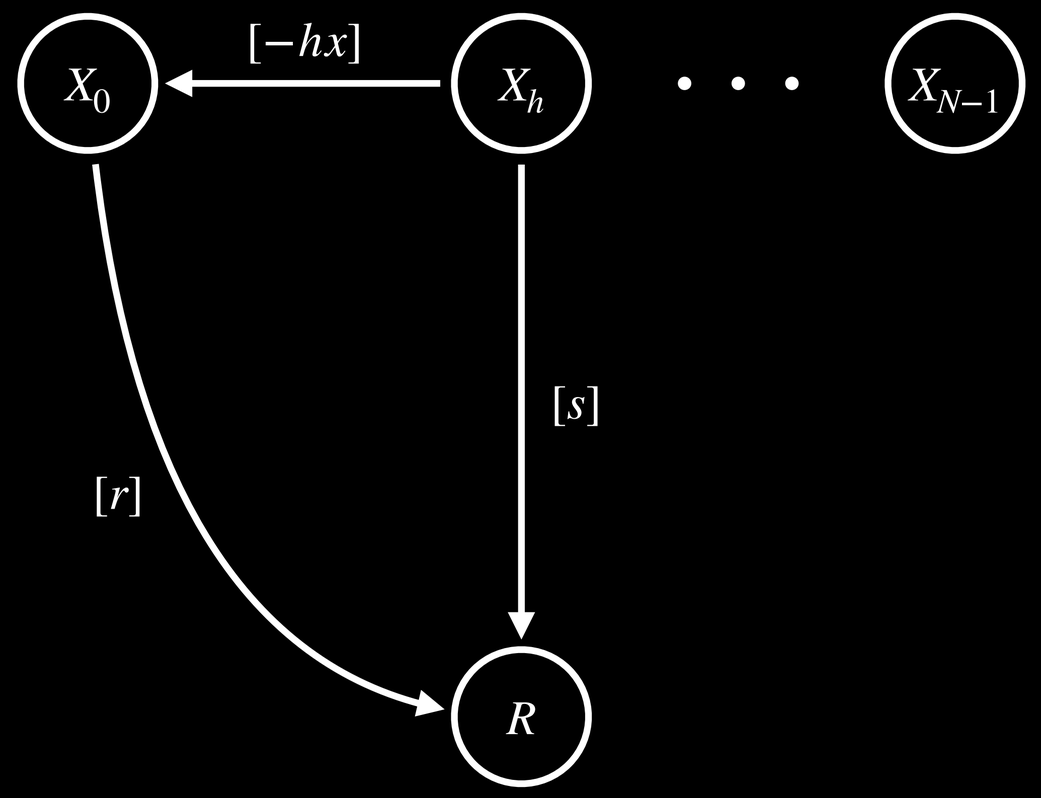
The Schnorr sigma protocol is a 3-message protocol to prove knowledge of a secret key x such that $X_1 = [x]X_0$, without revealing x.

1. **Commitment.** Prover samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and publishes $R = [r]X_0 = g^r$.

2. **Challenge.** Verifier publishes $h \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.

3. **Response.** Prover publishes $s = r - hx$.

Verifier accepts if $R = [s]X_h$.



Schnorr Signature Scheme

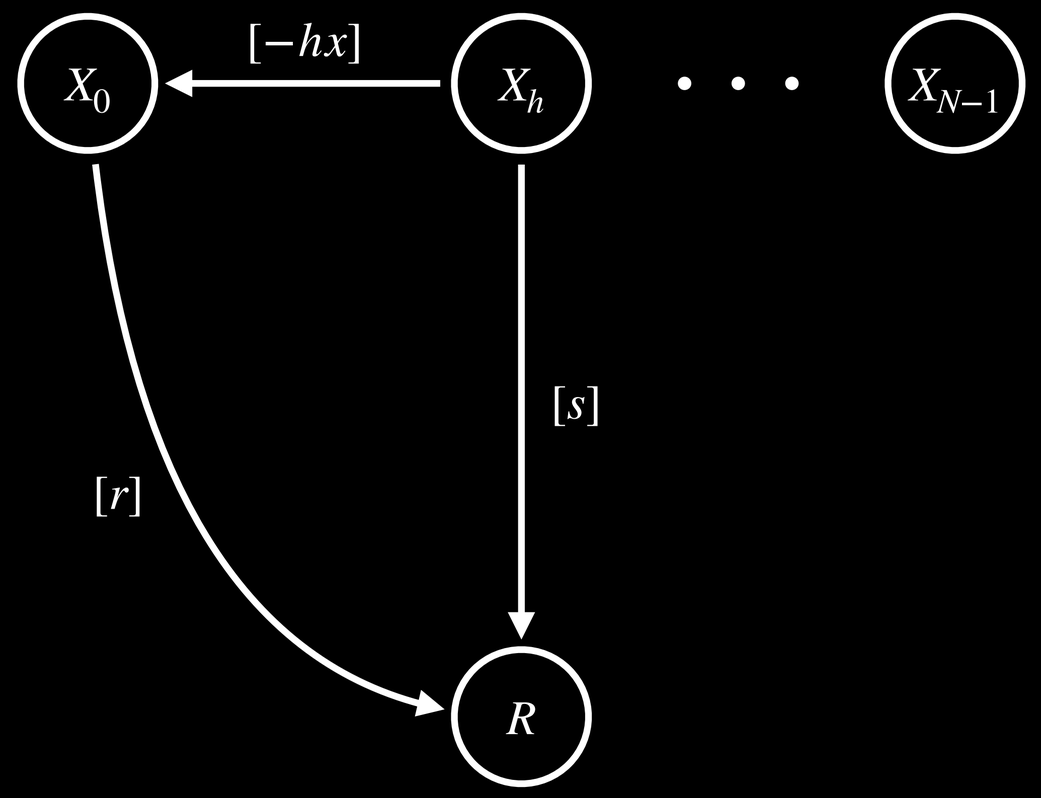
The **Fiat-Shamir transformation** replaces the verifier with a hash function H. To sign for message m with secret/public key (x, X_1):

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $R = [r]X_0 = g^r$.

2. **Challenge.** Signer computes $h = \mathrm{H}(m, R)$.

3. **Response.** Signer computes $s = r - hx$.

Signer outputs $\sigma = (R, s)$. Verifier computes $h = \mathrm{H}(m, R)$ and accepts if $R = [s]X_h$.



Schnorr Signature Scheme

The **Fiat-Shamir transformation** replaces the verifier with a hash function H. To sign for message m with secret/public key (x, X_1):

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $R = [r]X_0 = g^r$.
2. **Challenge.** Signer computes $h = \mathsf{H}(m, R)$.
3. **Response.** Signer computes $s = r - hx$.

Signer outputs $\sigma = (R, s)$. Verifier computes $h = \mathsf{H}(m, R)$ and accepts if $R = [s]X_h$.

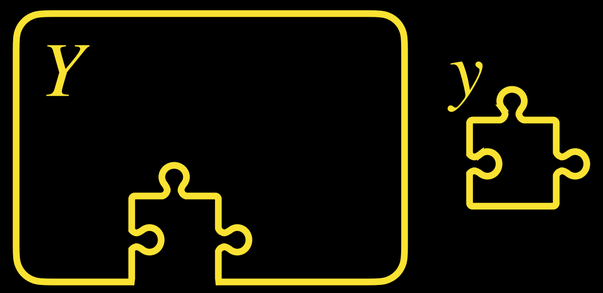
SECURITY

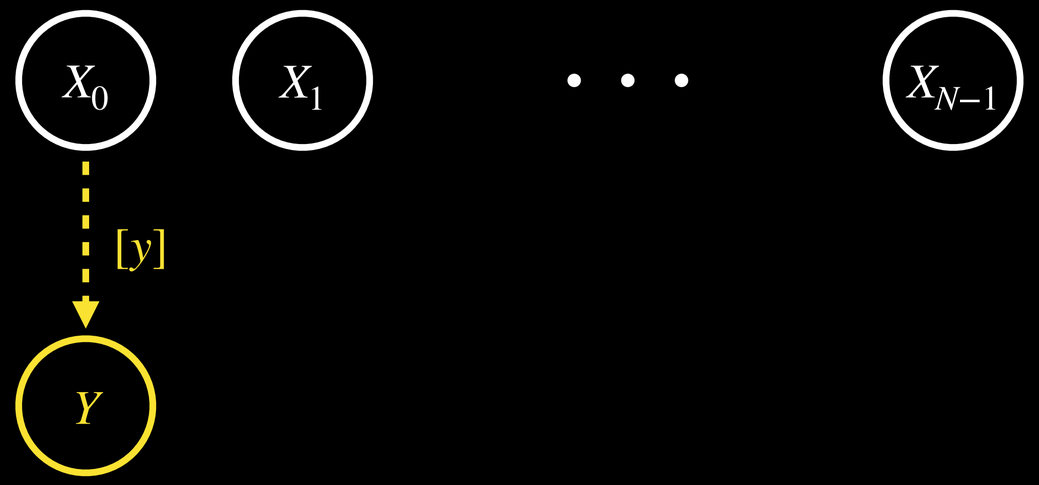
Thanks to the 2-special soundness and honest-verifier zero knowledge of the underlying sigma protocol, the Schnorr signature scheme is secure in the **random oracle model**.

Schnorr Adaptor Signatures Protocol

Setup

- Take a statement $Y = [y]X_0 = g^y$ for some unknown witness $y \in \mathbb{Z}/N\mathbb{Z}$.



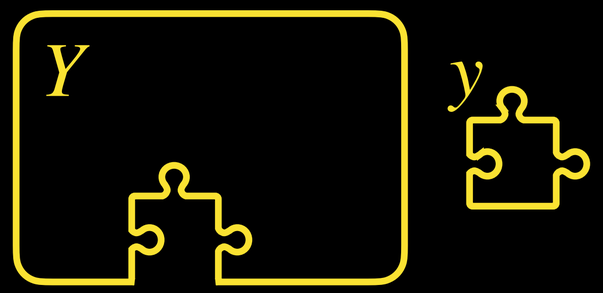


Schnorr Si[illegible]ocol

Adaptor signatures

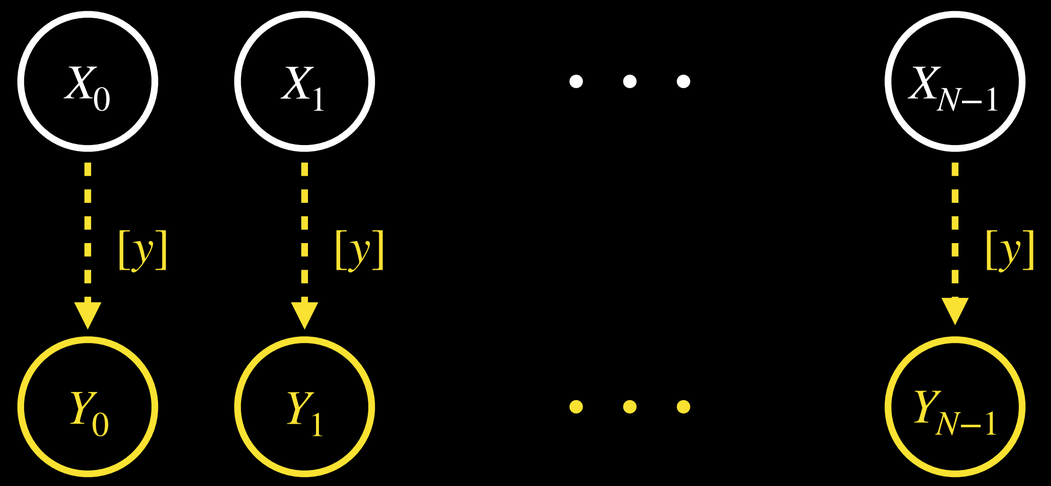
Setup

- Take a statement $Y = [y]X_0 = g^y$ for some unknown witness $y \in \mathbb{Z}/N\mathbb{Z}$.



- Define $Y_0 = Y$ and $Y_n = [y]X_n$.

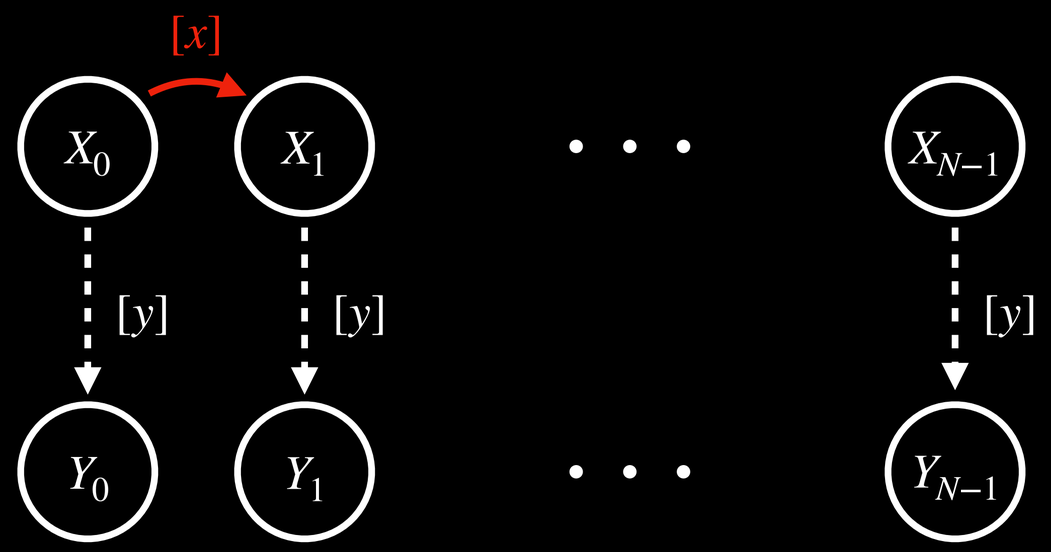
- ***Note: we can compute $Y_n = X_n \cdot Y$ from X_n and Y, without knowing either x or y.***



Schnorr Adaptor Signatures Protocol

Pre-Signing & Pre-Verification

To pre-sign for message m with respect to statement Y, with secret/public key (x, X_1):

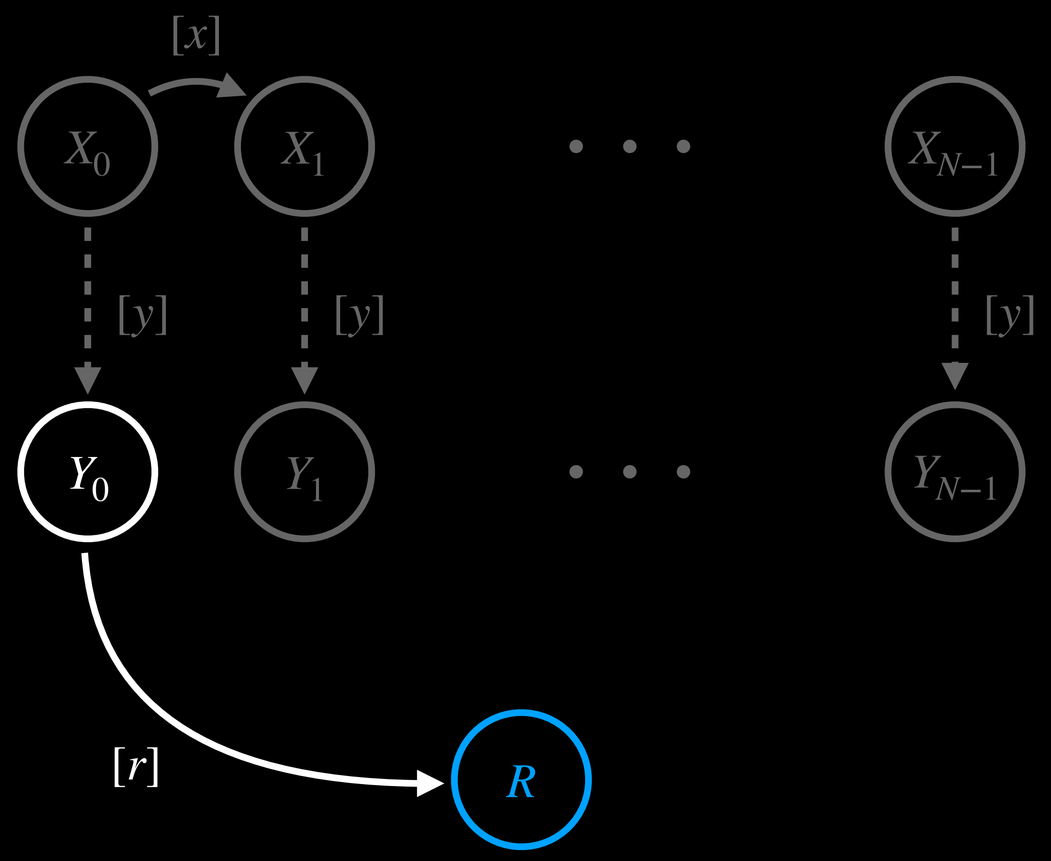


Schnorr Adaptor Signatures Protocol

Pre-Signing & Pre-Verification

To pre-sign for message m with respect to statement Y, with secret/public key (x, X_1):

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $R = [r]Y_0 = g^r \cdot Y$.



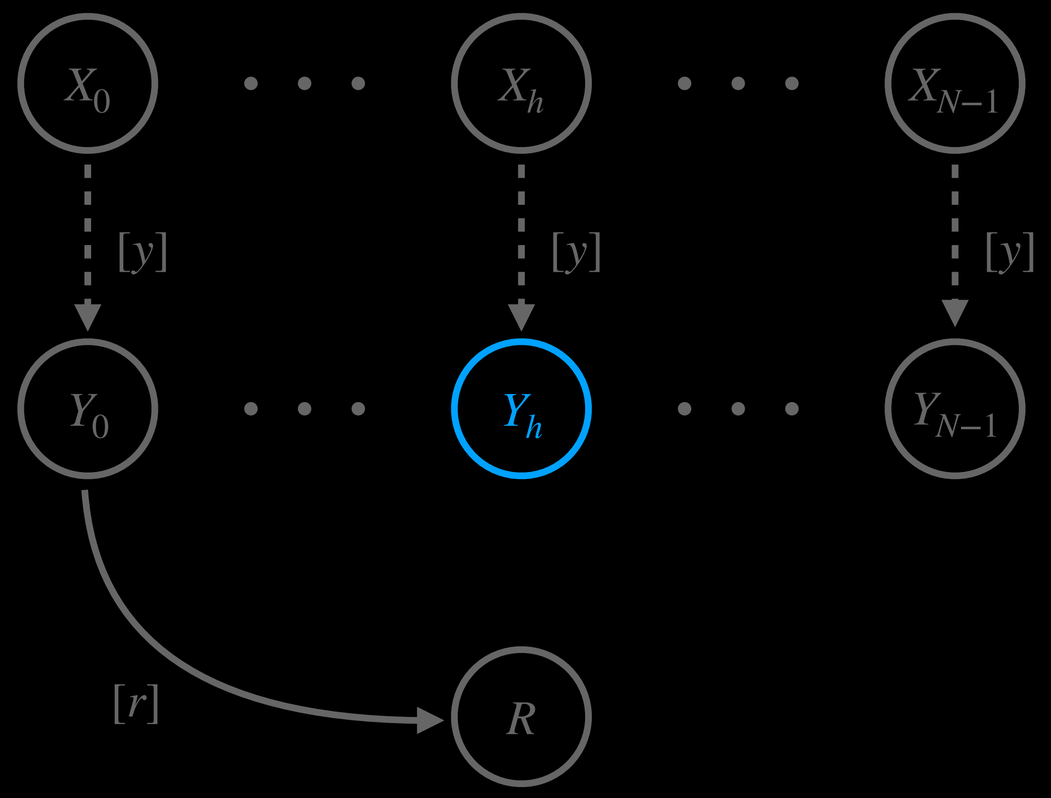
Schnorr Adaptor Signatures Protocol

Pre-Signing & Pre-Verification

To pre-sign for message m with respect to statement Y, with secret/public key (x, X_1):

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $R = [r]Y_0 = g^r \cdot Y$.

2. **Challenge.** Signer computes $h = \mathsf{H}(m, R)$.



Schnorr Adaptor Signatures Protocol

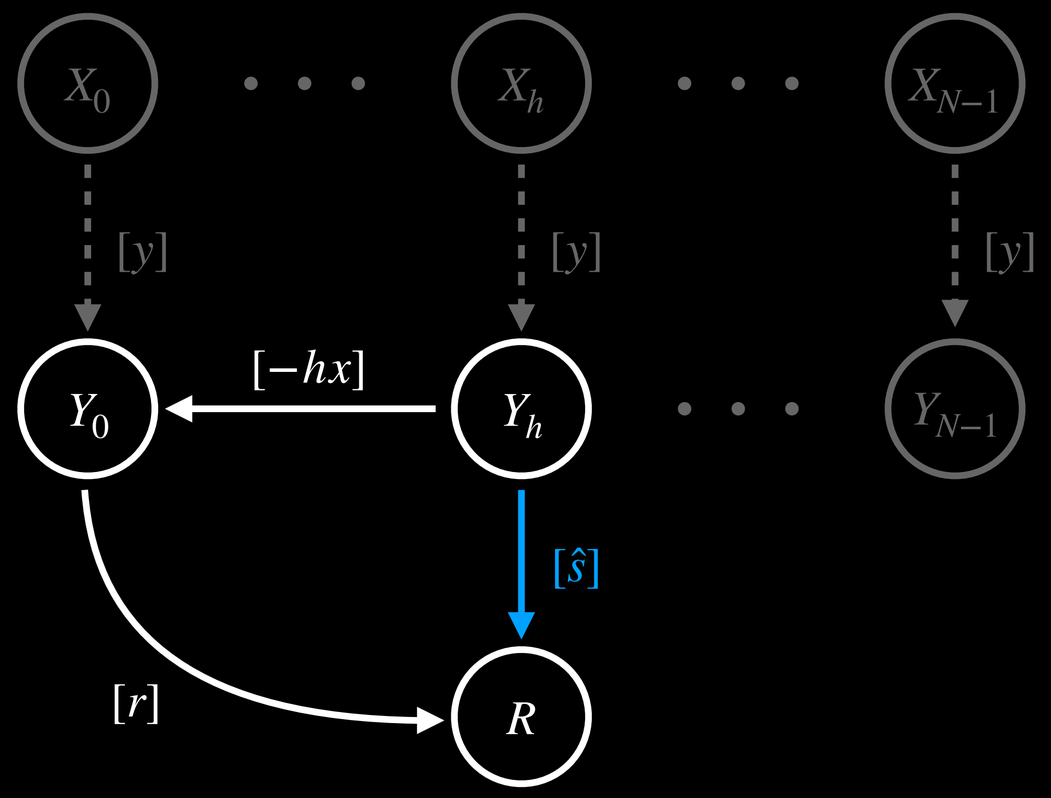
Pre-Signing & Pre-Verification

To pre-sign for message m with respect to statement Y, with secret/public key (x, X_1):

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $R = [r]Y_0 = g^r \cdot Y$.

2. **Challenge.** Signer computes $h = \mathsf{H}(m, R)$.

3. **Response.** Signer computes $\hat{s} = r - hx$.



Schnorr Adaptor Signatures Protocol

Pre-Signing & Pre-Verification

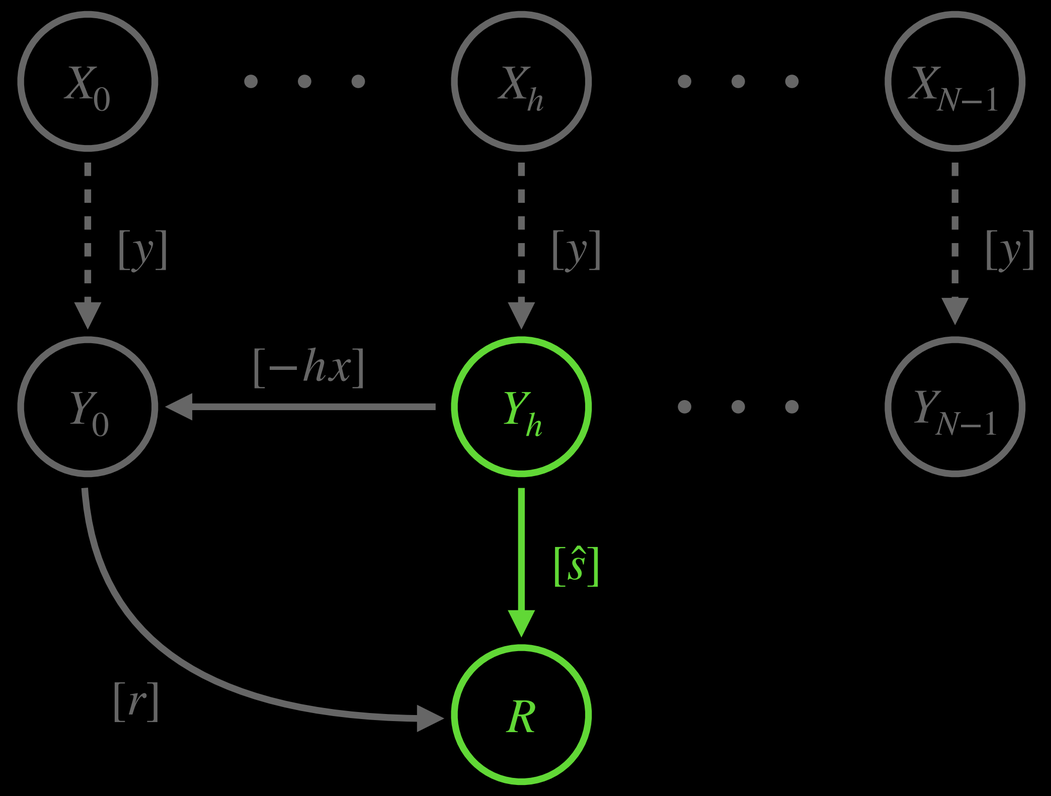
To pre-sign for message m with respect to statement Y, with secret/public key (x, X_1):

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $R = [r]Y_0 = g^r \cdot Y$.

2. **Challenge.** Signer computes $h = \mathsf{H}(m, R)$.

3. **Response.** Signer computes $\hat{s} = r - hx$.

Signer outputs $\hat{\sigma} = (R, \hat{s})$. Verifier computes $h = \mathsf{H}(m, R)$ and accepts if $R = [\hat{s}]Y_h$.



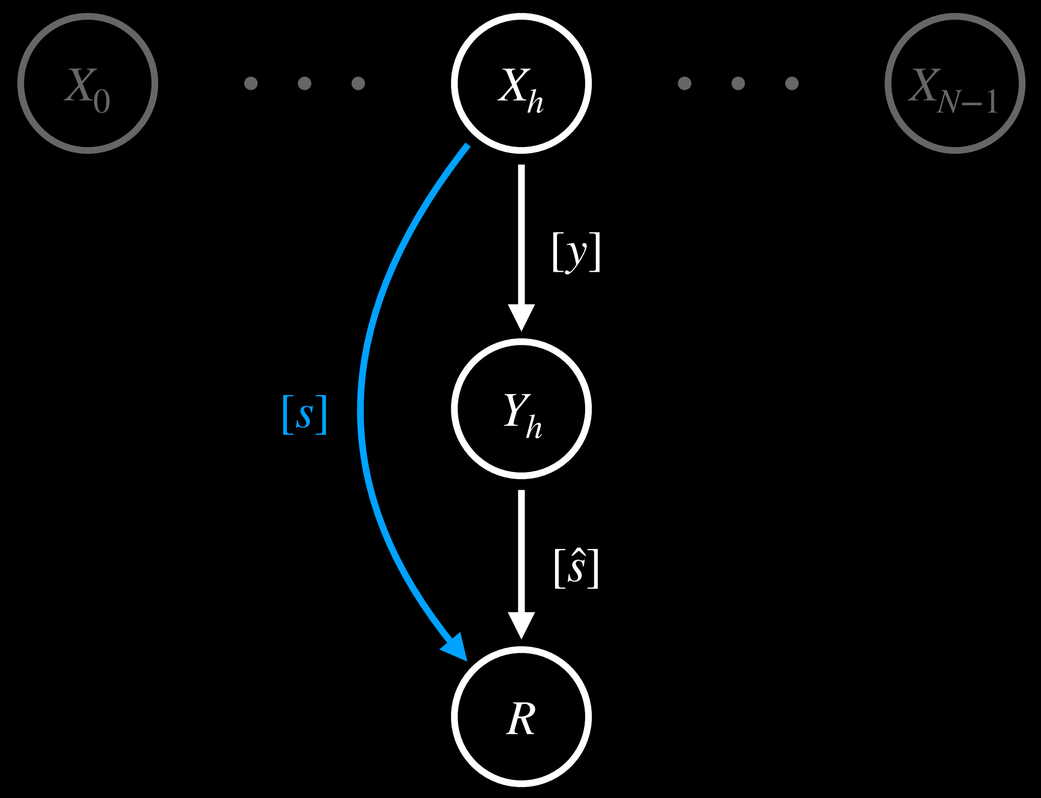
Schnorr Si[illegible]ocol

Adaptor signatures

Adapting

Use witness y to adapt pre-signature $(R, \hat{s})$ into signature (R, s) by computing $s = \hat{s} + y$.

If $Y_0 = [y]X_0$ *and* $R = [\hat{s}]Y_h$*, then* $R = [\hat{s} + y]X_h$.

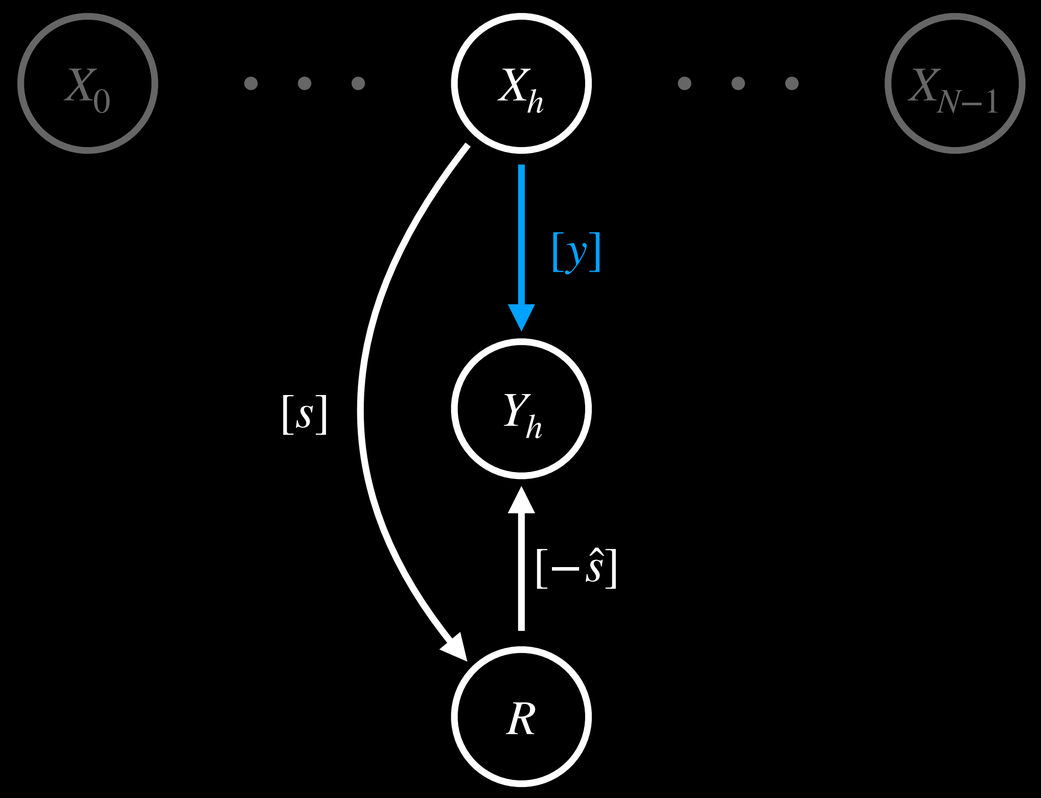


Schnorr Adaptor Signatures Protocol

Adapting

Use witness y to adapt pre-signature $(R, \hat{s})$ into signature (R, s) by computing $s = \hat{s} + y$.

If $Y_0 = [y]X_0$ *and* $R = [\hat{s}]Y_h$*, then* $R = [\hat{s} + y]X_h$.



Extracting

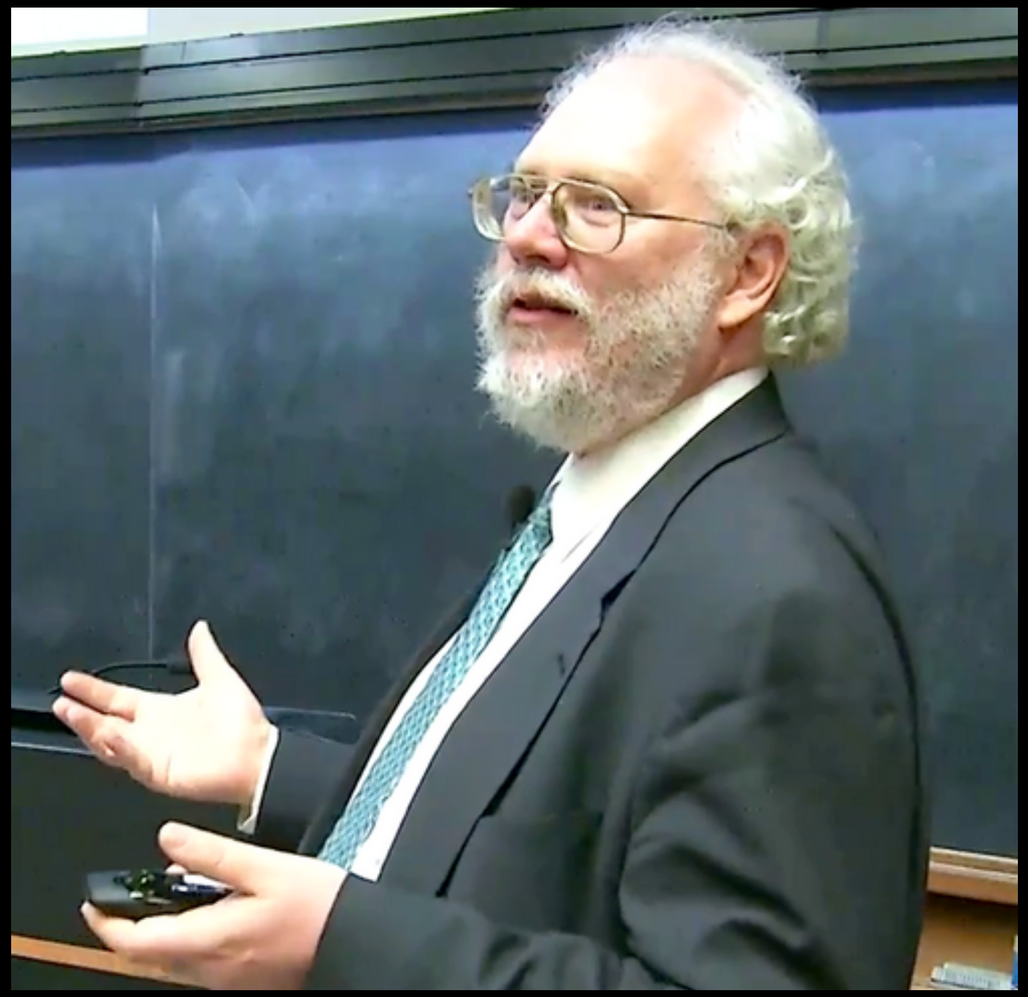
Extract witness y from pre-signature $(R, \hat{s})$ and adapted signature (R, s) by computing $y = s - \hat{s}$.

If $[\hat{s}]Y_h = R = [s]X_h$*, then* $Y_0 = [s - \hat{s}]X_0$.

Too Good to be True?

Shor's Algorithm (1994) allows quantum computers to solve the discrete log problem efficiently, breaking Schnorr signatures.

Can we recreate this approach using quantum-safe tools?



III. CSI-FiSh

Post-Quantum Analog of Schnorr

CSI-FiSh Signatures

Setup

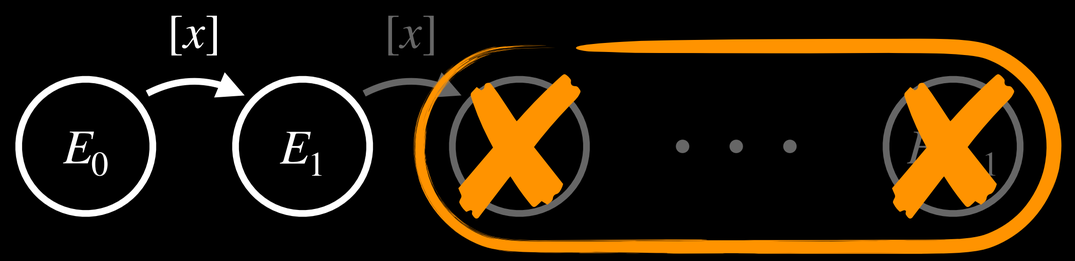
- Let E_0 be a supersingular elliptic curve defined over $\mathbb{F}_p$, with $\mathrm{End}_p(E_0) \cong \mathcal{O}$.
- Let X be the set of supersingular elliptic curves E defined over $\mathbb{F}_p$, with $\mathrm{End}_p(E) \cong \mathcal{O}$.
- There is a group action of $\mathbb{Z}/N\mathbb{Z}$ on X given by the action of the ideal class group $\mathrm{cl}(\mathcal{O})$.
- Pick a secret key $x \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.
- Define $E_n = [nx]E_0$ for each $n \in \mathbb{Z}/N\mathbb{Z}$.

$$E_0 \xrightarrow{[x]} E_1 \xrightarrow{[x]} E_2 \quad \cdots \quad E_{N-1}$$

CSI-FiSh Signatures

Setup

- Let E_0 be a supersingular elliptic curve defined over $\mathbb{F}_p$, with $\mathrm{End}_p(E_0) \cong \mathcal{O}$.
- Let X be the set of supersingular elliptic curves E defined over $\mathbb{F}_p$, with $\mathrm{End}_p(E) \cong \mathcal{O}$.
- There is a group action of $\mathbb{Z}/N\mathbb{Z}$ on X given by the action of the ideal class group $\mathrm{cl}(\mathcal{O})$.
- Pick a secret key $x \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.
- Define $E_n = [nx]E_0$ for each $n \in \mathbb{Z}/N\mathbb{Z}$.



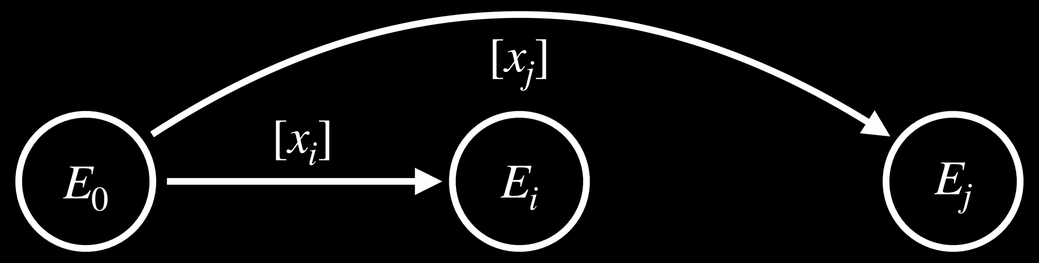
PROBLEM!

The verifier will need to compute E_h, but since X is a set, not a group, we cannot efficiently compute E_i from E_1 without knowing x.

CSI-FiSh Signatures

Setup

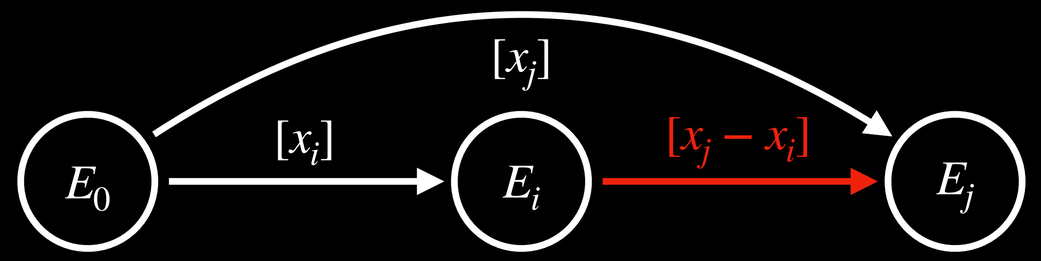
- Let E_0 be a supersingular elliptic curve defined over $\mathbb{F}_p$, with $\mathrm{End}_p(E_0) \cong \mathcal{O}$.
- Let X be the set of supersingular elliptic curves E defined over $\mathbb{F}_p$, with $\mathrm{End}_p(E) \cong \mathcal{O}$.
- There is a group action of $\mathbb{Z}/N\mathbb{Z}$ on X given by the action of the ideal class group $\mathrm{cl}(\mathcal{O})$.
- Pick secret keys $x_1, \ldots, x_{n-1} \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.
- Define $x_0 = 0$ and $E_i = [x_i]E_0$.



CSI-FiSh Signatures

Setup

- Let E_0 be a supersingular elliptic curve defined over $\mathbb{F}_p$, with $\mathrm{End}_p(E_0) \cong \mathcal{O}$.
- Let X be the set of supersingular elliptic curves E defined over $\mathbb{F}_p$, with $\mathrm{End}_p(E) \cong \mathcal{O}$.
- There is a group action of $\mathbb{Z}/N\mathbb{Z}$ on X given by the action of the ideal class group $\mathrm{cl}(\mathcal{O})$.
- Pick secret keys $x_1, \ldots, x_{n-1} \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$.
- Define $x_0 = 0$ and $E_i = [x_i]E_0$.



ASSUMPTION

Given $(E_0, \ldots, E_n)$, the **multi-target group action inverse problem** is to find (i, j, x_{ij}) such that $E_j = [x_{ij}]E_i$. We assume this problem is infeasible.

CSI-FiSh Signatures

Signing & Verification

To sign for message m with secret/public key $(x_1, \ldots, x_{n-1}, E_1, \ldots, E_{n-1})$, we follow Schnorr:

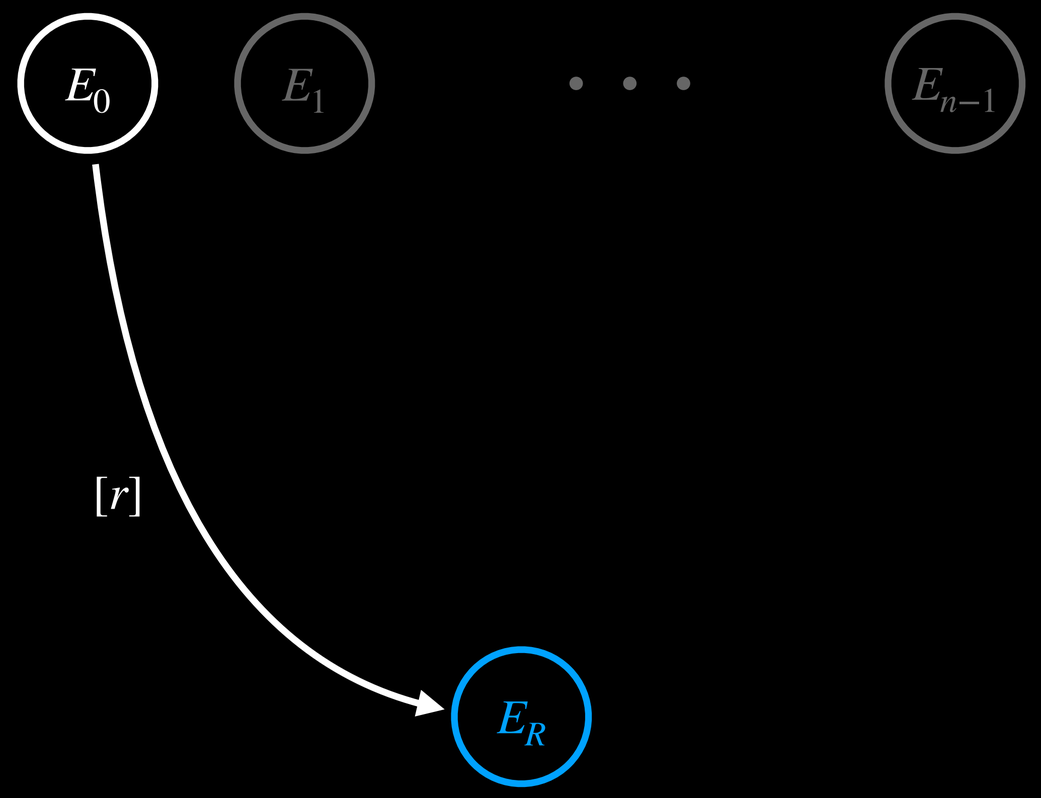
E_0 E_1 $\cdots$ E_{n-1}

CSI-FiSh Signatures

Signing & Verification

To sign for message m with secret/public key $(x_1, \ldots, x_{n-1}, E_1, \ldots, E_{n-1})$, we follow Schnorr:

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $E_R = [r]E_0$.



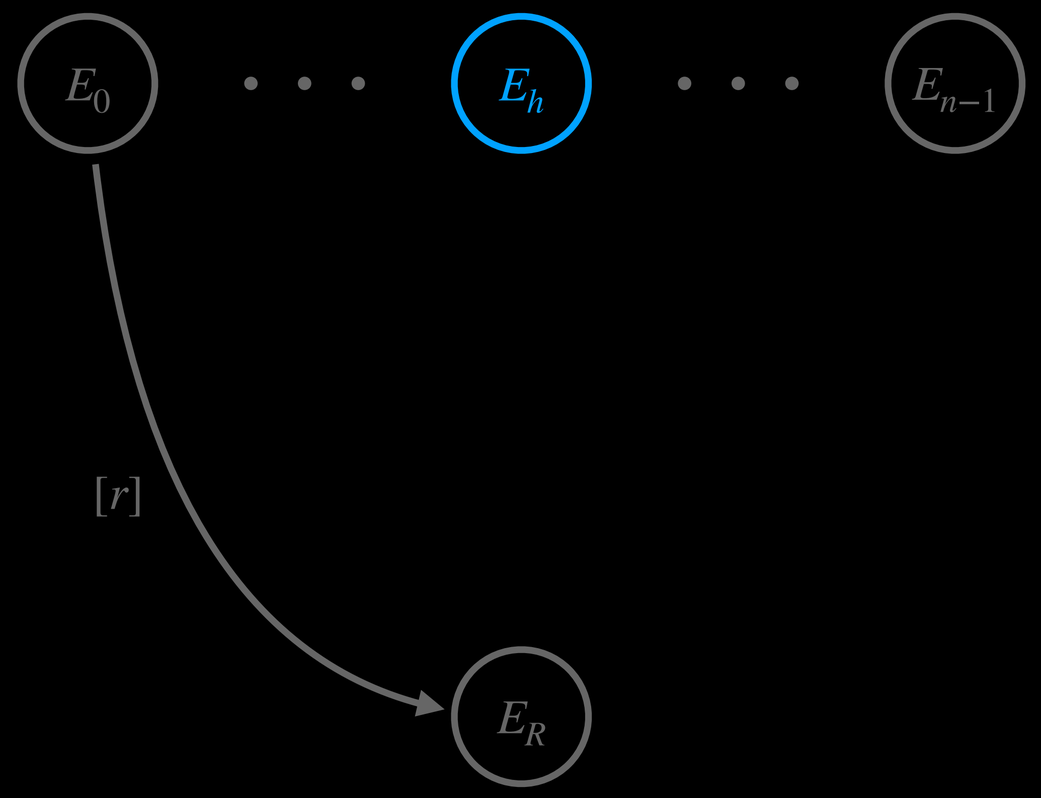
CSI-FiSh Signatures

Signing & Verification

To sign for message m with secret/public key $(x_1, \ldots, x_{n-1}, E_1, \ldots, E_{n-1})$, we follow Schnorr:

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $E_R = [r]E_0$.

2. **Challenge.** Signer computes $h = \mathsf{H}(m, E_R)$.



CSI-FiSh Signatures

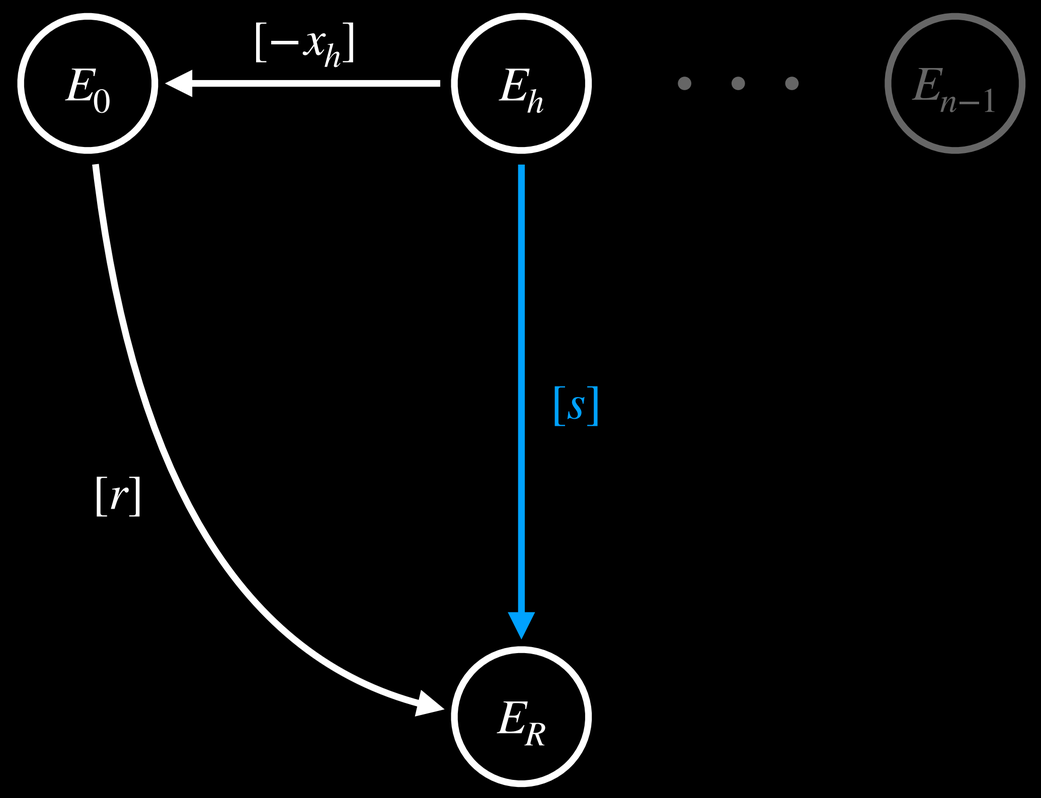
Signing & Verification

To sign for message m with secret/public key $(x_1, \ldots, x_{n-1}, E_1, \ldots, E_{n-1})$, we follow Schnorr:

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $E_R = [r]E_0$.

2. **Challenge.** Signer computes $h = \mathsf{H}(m, E_R)$.

3. **Response.** Signer computes $s = r - x_h$.



CSI-FiSh Signatures

Signing & Verification

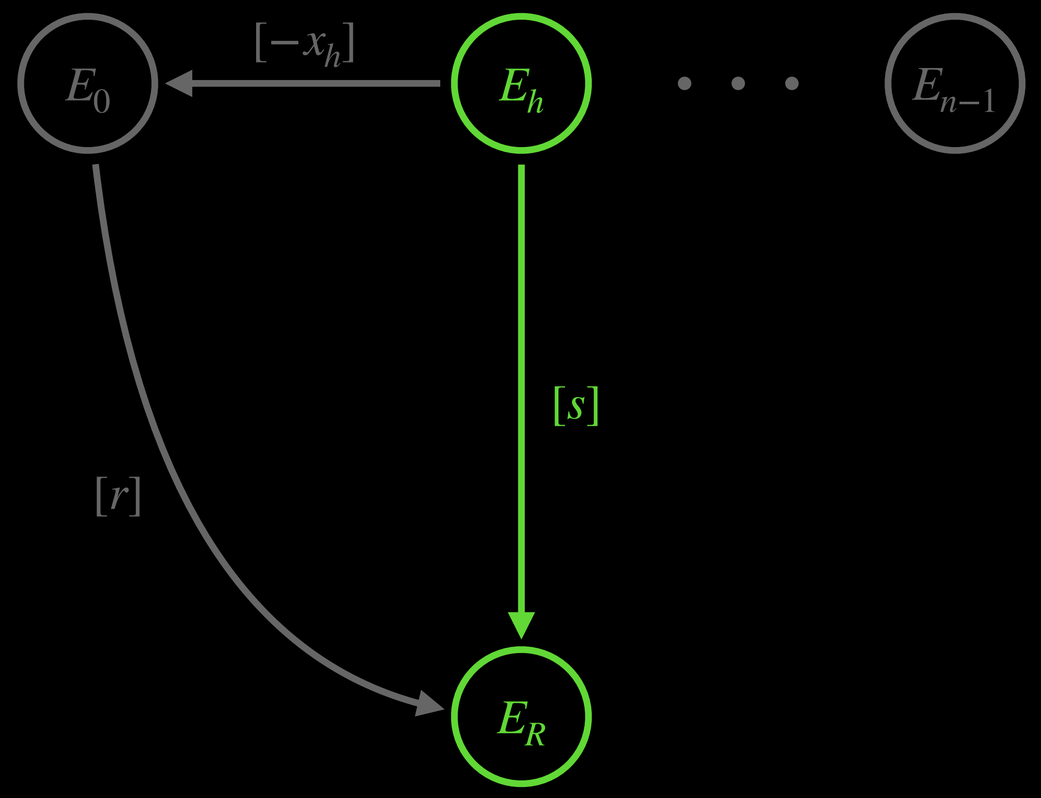
To sign for message m with secret/public key $(x_1, \ldots, x_{n-1}, E_1, \ldots, E_{n-1})$, we follow Schnorr:

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $E_R = [r]E_0$.

2. **Challenge.** Signer computes $h = \mathsf{H}(m, E_R)$.

3. **Response.** Signer computes $s = r - x_h$.

Signer outputs $\sigma = (E_R, s)$. Verifier computes $h = \mathsf{H}(m, E_R)$ and accepts if $E_R = [s]E_h$.



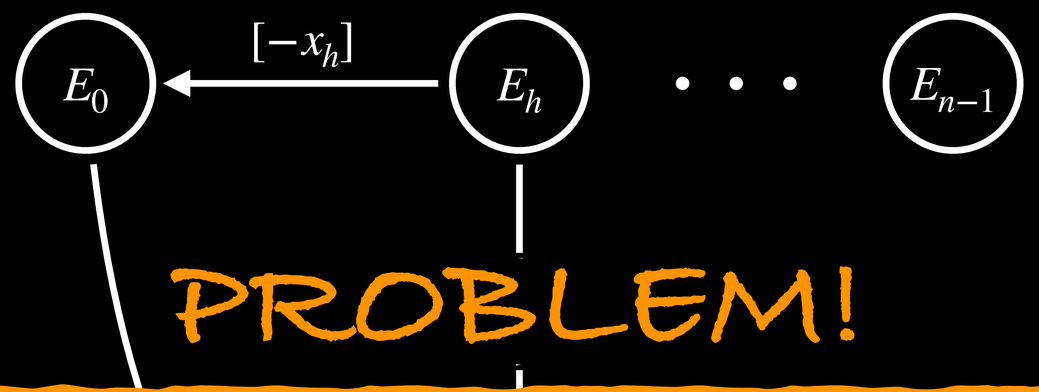
CSI-FiSh Signatures

Signing & Verification

To sign for message m with secret/public key $(x_1, \ldots, x_{n-1}, E_1, \ldots, E_{n-1})$, we follow Schnorr:

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $E_R = [r]E_0$.
2. **Challenge.** Signer computes $h = \mathsf{H}(m, E_R)$.
3. **Response.** Signer computes $s = r - x_h$.

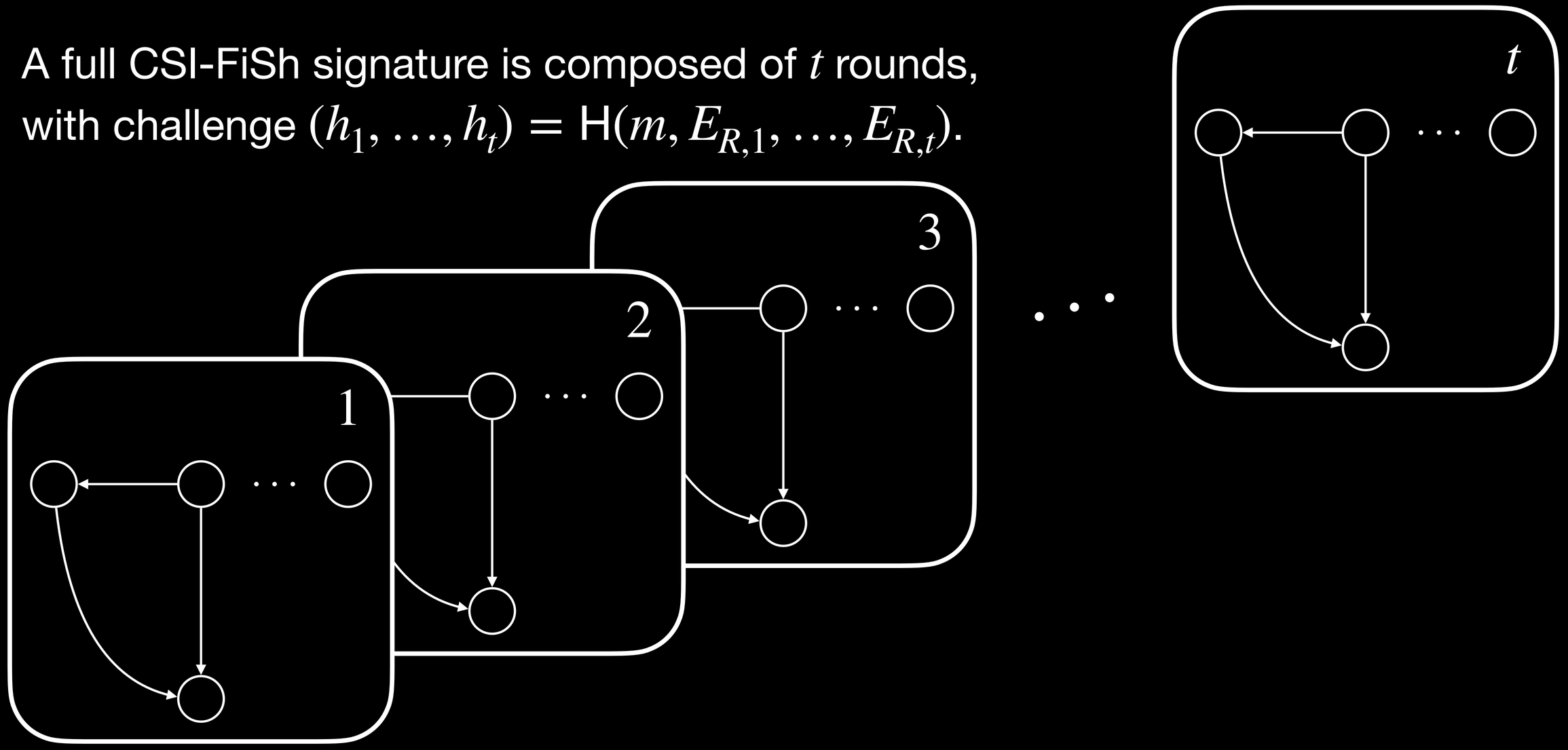
Signer outputs $\sigma = (E_R, s)$. Verifier computes $h = \mathsf{H}(m, E_R)$ and accepts if $E_R = [s]E_h$.



The size of the challenge space is no longer N but n. We must repeat for t rounds, where $n^t \approx N$, to make the challenge space large enough.

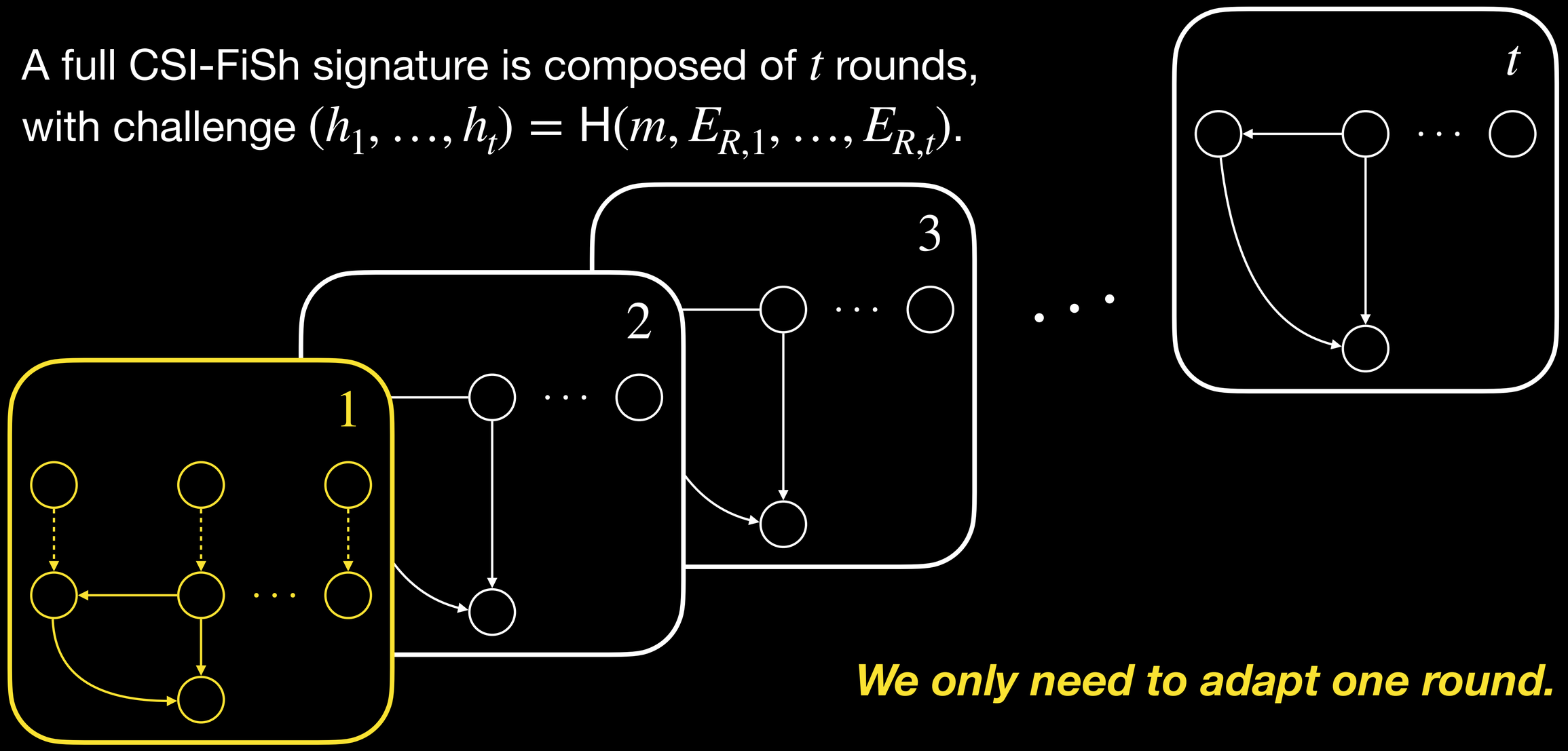
Multiple Rounds

A full CSI-FiSh signature is composed of t rounds, with challenge $(h_1, \ldots, h_t) = \mathsf{H}(m, E_{R,1}, \ldots, E_{R,t})$.



Multiple Rounds

A full CSI-FiSh signature is composed of t rounds, with challenge $(h_1, \ldots, h_t) = \mathsf{H}(m, E_{R,1}, \ldots, E_{R,t})$.



We only need to adapt one round.

IV. ORCAS

Adaptor Signatures for CSI-FiSh

Adaptor CSI-Fish: Take 1

Pre-Signing & Pre-Verification

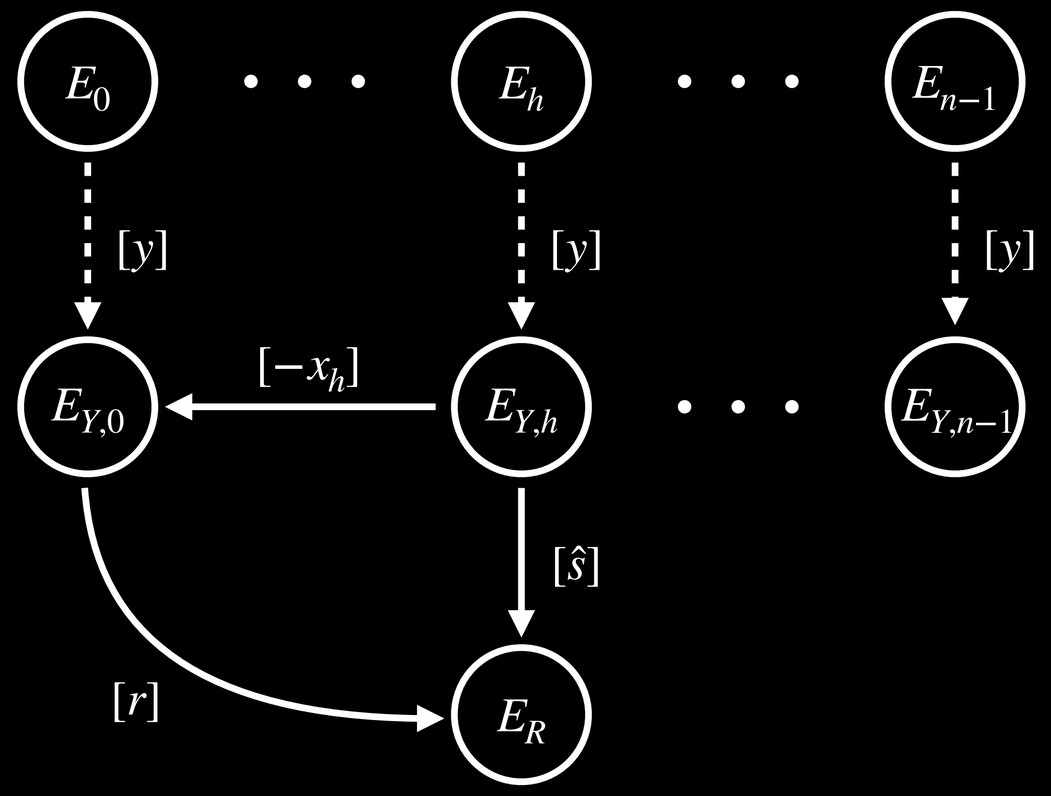
To pre-sign for message m w.r.t. statement E_Y and with keys $(x_1, \ldots, x_{n-1}, E_1, \ldots, E_{n-1})$:

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $E_R = [r]E_{Y,0}$.

2. **Challenge.** Signer computes $h = \mathsf{H}(m, E_R)$.

3. **Response.** Signer computes $\hat{s} = r - x_h$.

Signer outputs $\hat{\sigma} = (E_R, \hat{s})$. Verifier computes $h = \mathsf{H}(m, E_R)$ and accepts if $E_R = [\hat{s}]E_{Y,h}$.



Adaptor CSI-Fish: Take 1

Pre-Signing & Pre-Verification

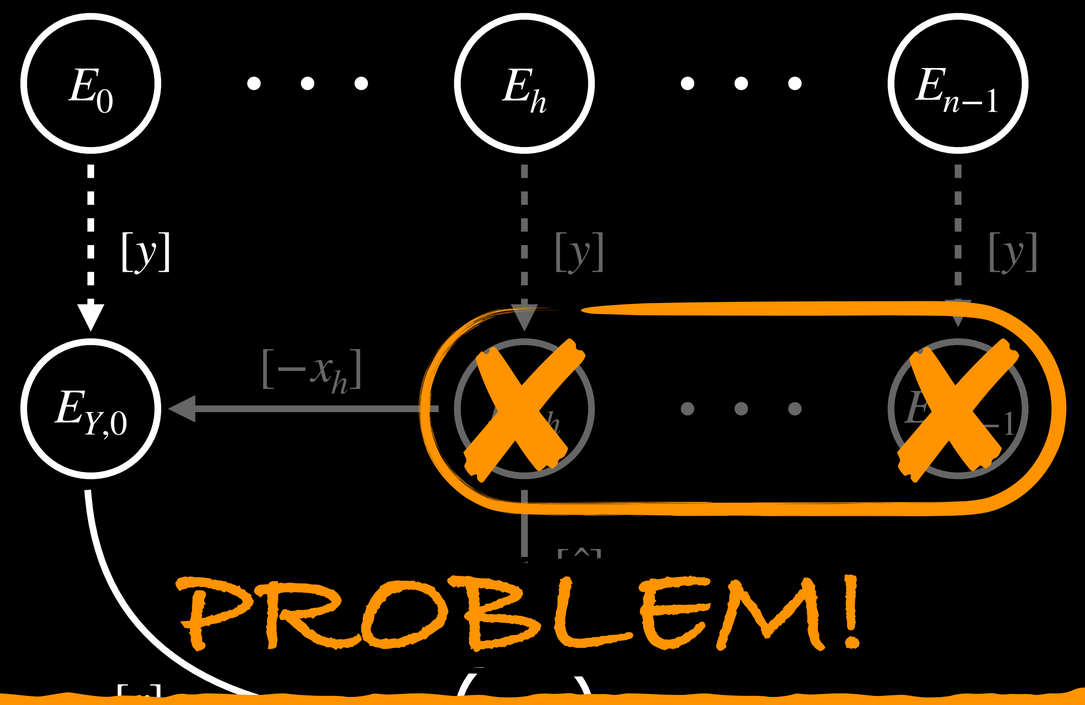
To pre-sign for message m w.r.t. statement E_Y and with keys $(x_1, \ldots, x_{n-1}, E_1, \ldots, E_{n-1})$:

1. **Commitment.** Signer samples $r \xleftarrow{\$} \mathbb{Z}/N\mathbb{Z}$ and computes $E_R = [r]E_{Y,0}$.

2. **Challenge.** Signer computes $h = \mathsf{H}(m, E_R)$.

3. **Response.** Signer computes $\hat{s} = r - x_h$.

Signer outputs $\hat{\sigma} = (E_R, \hat{s})$. Verifier computes $h = \mathsf{H}(m, E_R)$ and accepts if $E_R = [\hat{s}]E_{Y,h}$.



The verifier has no way to compute $E_{Y,h}$ for $h \neq 0$ without knowing y.

Three Solutions

1. Tairi *et al.* (2021) avoid this problem using non-interactive zero-knowledge proofs.
   + “Honest” adaptor signature scheme for CSI-FiSh
   – Large pre-signatures and slow pre-signing

Three Solutions

1. Tairi *et al.* (2021) avoid this problem using non-interactive zero-knowledge proofs.
   + “Honest” adaptor signature scheme for CSI-FiSh
   – Large pre-signatures and slow pre-signing

2. Jana *et al.* (2024) add an extra round to CSI-FiSh which always has $h = 0$.
   + Fast pre-signing and short pre-signatures
   – Incompatible with standard CSI-FiSh

Three Solutions

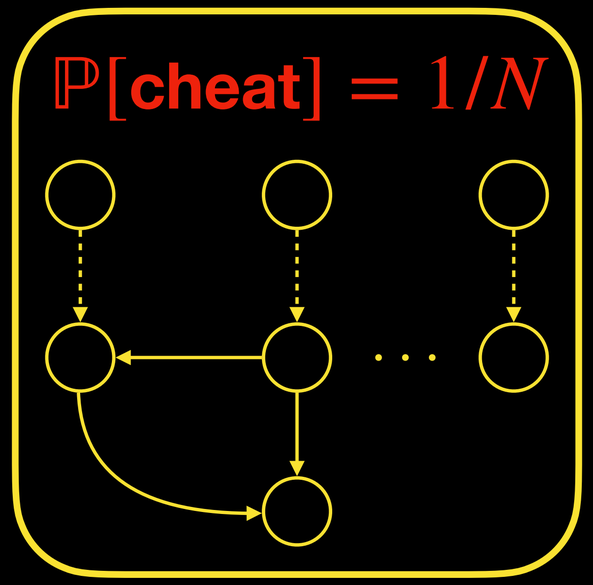
1. Tairi *et al.* (2021) avoid this problem using non-interactive zero-knowledge proofs.
   + "Honest" adaptor signature scheme for CSI-FiSh
   – Large pre-signatures and slow pre-signing

2. Jana *et al.* (2024) add an extra round to CSI-FiSh which always has $h = 0$.
   + Fast pre-signing and short pre-signatures
   – Incompatible with standard CSI-FiSh

3. We "cheat" for this one round, repeating until we receive challenge $h = 0$.
   + Short pre-signatures and (generally) fast pre-signing
   + Adapted pre-signatures are genuine CSI-FiSh signatures
   – Slow pre-signing for certain parameter choices

One Round Cheating

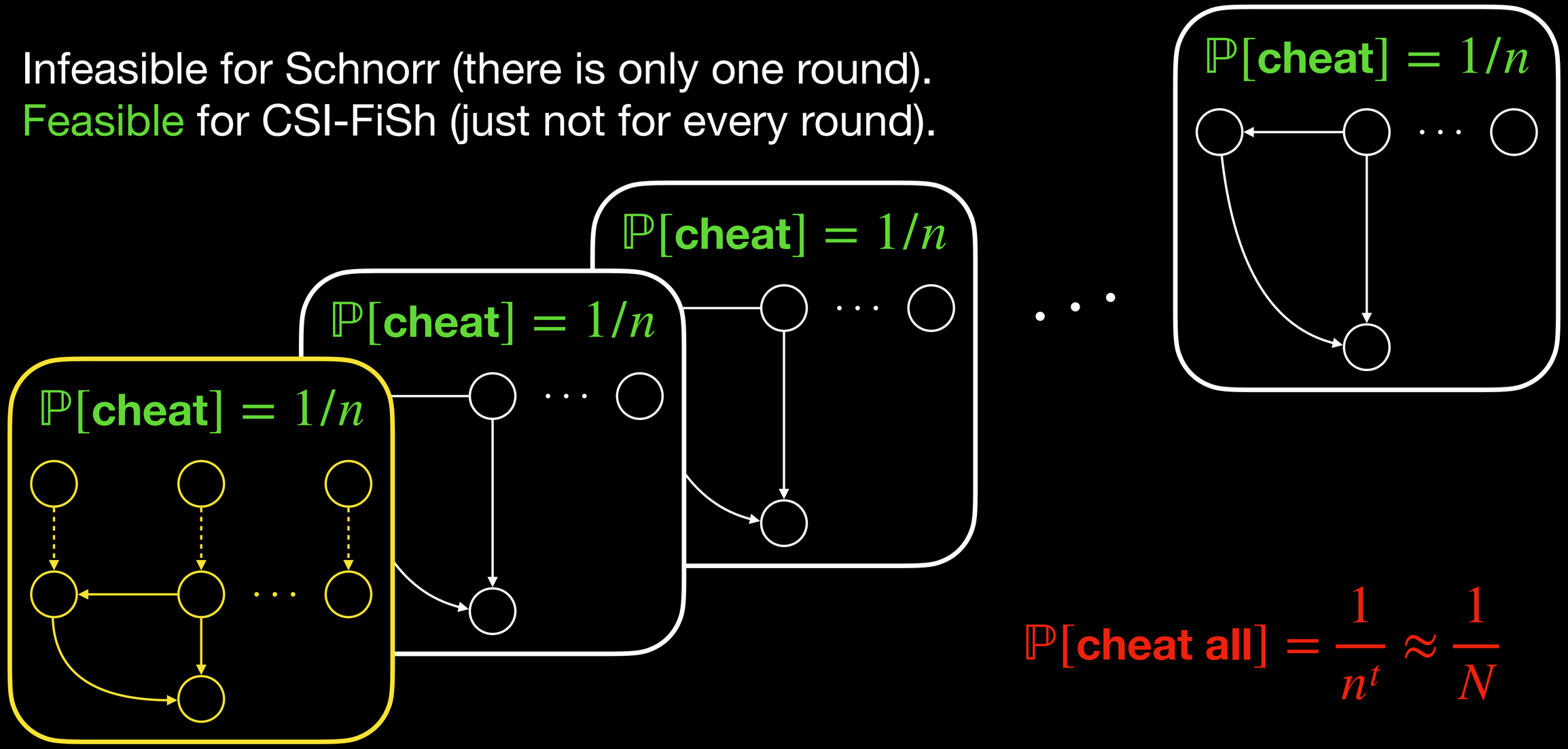
Infeasible for Schnorr (there is only one round).



$$\mathbb{P}[\textbf{cheat all}] = \frac{1}{N}$$

One Round Cheating

Infeasible for Schnorr (there is only one round).
Feasible for CSI-FiSh (just not for every round).



$$\mathbb{P}[\textbf{cheat all}] = \frac{1}{n^t} \approx \frac{1}{N}$$

Parameters & Optimizations

- CSI-FiSh uses 2^d public keys E_i (for varying values of d).
- Due to *quadratic twists*, the challenge space is $\{-2^d, \ldots, 0, \ldots, 2^d\}$ for each round. On average pre-signing has to repeat 2^{d+1} times.
- CSI-FiSh uses a *slow hash function* with rate k, which takes 2^k normal hash operations to perform a single hash.
- ORCAS re-randomizes the commitments by permuting them, rather than generating them again from scratch. This is much faster!

d	t	k	|sk|	|pk|	$\|\widehat{\sigma}\|$	$\|\sigma\|$	KeyGen	Sig	Ver	PreSig	PreVer	Adapt	Ext
1	49	15	16	128	1650	1649	0.11	2.00	2.01	2.03	2.00	0.005	0.005
2	36	14	16	256	1221	1220	0.19	1.46	1.47	1.49	1.47	0.005	0.005
3	28	14	16	512	957	956	0.36	1.15	1.15	1.22	1.15	0.005	0.005
4	23	12	16	1024	792	791	0.70	0.94	0.95	0.99	0.95	0.005	0.005
5	19	14	16	2048	660	659	1.27	0.78	0.78	1.14	0.78	0.005	0.005
6	16	16	16	4096	561	560	2.65	0.68	0.68	3.40	0.68	0.005	0.005
7	14	16	16	8192	495	494	5.28	0.59	0.59	6.93	0.59	0.005	0.005
8	13	11	16	16384	462	461	10.39	0.53	0.53	0.85	0.54	0.005	0.005
9	12	8	16	32768	429	428	20.55	0.49	0.49	0.61	0.49	0.005	0.005
10	11	7	16	65536	396	395	40.67	0.45	0.45	0.55	0.44	0.005	0.005
11	10	8	16	131072	363	362	81.09	0.40	0.40	0.77	0.40	0.005	0.005
12	9	11	16	262144	330	329	164.38	0.37	0.36	6.23	0.36	0.005	0.005

Table 1. ORCAS performance. Sizes are in bytes and times in seconds.

Benchmark Results

Parameters: 2^d public keys, t rounds, 2^k times slower hash

Pre-signing overhead scales as approximately 2^{d+k}.

Thank you!

Questions?